CENG 111 Algoritmalar ve Programlama 4: Karar ve Kontrol İfadeleri

Öğr.Gör. Şevket Umut ÇAKIR

Pamukkale Üniversitesi

Hafta 4

Anahat

1 Algoritma

Algoritma

- Hesaplama problemlerinin çözümü belli adımların sırayla yapılmasını gerektirir
- **Algoritma** problemleri çözme prosedürüdür ve
  - Gerçekleştirilecek **eylemleri** ve
  - bu eylemlerin **sırasını** belirler
- İşlemlerin hangi sırada yapılacağı önemlidir
- Sabah uyanma ve işe gitme algoritması:
  1. Yataktan kalk
  2. Pijamalarını çıkart
  3. Duş al
  4. Giyin
  5. Kahvaltını yap
  6. İşe git

Algoritma

- İşe ıslak gitme algoritması
  1. Yataktan kalk
  2. Pijamalarını çıkart
  3. Giyin
  4. Duş al
  5. Kahvaltını yap
  6. İşe git

Pseudocode(Sözde/Sahte Kod)

- Pseudocode, eylemleri C koduna dönüştürmeden önce günlük kullanıma benzer ifadeler içerir
- Programı yazmadan önce programı düşünmemizi sağlar
- Metin düzenleyicide yazılabilir
- Sahte kod, sahte kodu C'ye dönüştürüp programı çalıştırdığınızda yürütülecek eylemleri ve kararları açıklar
- Örneğin `int i = 0;` ifadesi derleyiciye i değişkeninin türünü ve ilk değerini belirtir, eylem içermez

Kontrol Yapıları - Giriş

- Programlarda ifadeler genellikle yazıldığı sırayla çalıştırılır.
- Bu duruma **ardışık yürütme** denir.
- C'de, sıradaki yerine başka bir ifadenin çalışmasını sağlayan yapılar vardır.
- Bu duruma **kontrol aktarımı** denir.
- 1960'larda, kontrol aktarımının yanlış kullanımı yazılım geliştirme zorluklarına yol açtı.
- Bu sorunlar genellikle `goto` ifadesiyle ilişkilendirildi.
- **Yapısal programlama**, `goto`'nun ortadan kaldırılmasına yönelik bir yaklaşımdır.

Yapısal Programlama ve Kontrol Yapıları

- Böhm ve Jacopini, programların `goto` ifadesi olmadan yazılabileceğini gösterdi.
- Programlar, üç temel kontrol yapısıyla yazılabilir:
  - **Sıralı yapı**: İfadeler yazıldıkları sırayla çalıştırılır.
  - **Seçim yapısı**: Bir koşula bağlı olarak farklı ifadeler çalıştırılır.
  - **Döngü yapısı**: Belirli bir koşul sağlandığı sürece tekrar eden işlemler.
- Yapısal programlama, programların daha açık ve hata ayıklanabilir olmasını sağlar.
- Akış diyagramları ilk hafta işlenmiştir

C Seçim ve Döngü Komutları

- C dili seçim için üç yapı bulundurur
  - **Tekli `if` komutu**: Koşul *doğru* ise çalışacak işlemi/işlemleri seçer
  - `if ... else` **çift seçim komutu**: Koşul *doğru* ise bir işlemi, koşul *yanlış* ise diğer işlem(leri) seçer
  - `switch` **çoklu seçme komutu**: Bir ifadenin sonucuna göre bir çok işlemden birini seçer
- Döngü için `while`, `do...while` ve `for` gibi komutlar bulunur, işlemleri tekrarlı olarak yapar
- C dili toplam yedi kontrol ifadesi içerir, programlar bunların kombinasyonları ile oluşturulur

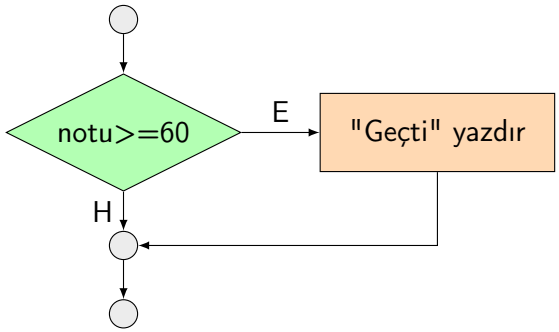
if Seçme Komutu

- Pseudocode

```
Eğer öğrencinin notu 60 veya üzerindeyse
        "Geçti" yazdır
```

- C kodu

```c
if (notu >= 60) {
        puts("Geçti");
}
```



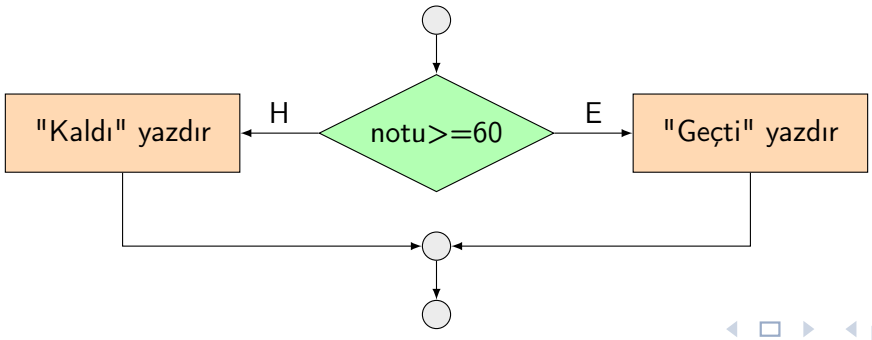
if...else Seçme Komutu

- Pseudocode

```
Eğer öğrencinin notu 60 veya üzerindeyse
        "Geçti" yazdır
Aksi halde
        "Kaldı" yazdır
```

- C kodu

```c
if (notu >= 60) {
        puts("Geçti");
} // if sonu
else {
        puts("Kaldı");
}
```



İç içe `if` ifadeleri

```
If student's grade is greater than or equal to 90
    Print "A"
else
    If student's grade is greater than or equal to 80
        Print "B"
    else
        If student's grade is greater than or equal to 70
            Print "C"
        else
            If student's grade is greater than or equal to 60
                Print "D"
            else
                Print "F"
```

İç içe `if` ifadeleri

```
if (grade >= 90) {
   puts("A");
} // end if
else {
   if (grade >= 80) {
      puts("B");
   } // end if
   else {
      if (grade >= 70) {
         puts("C");
      } // end if
      else {
         if (grade >= 60) {
            puts("D");
         } // end if
         else {
            puts("F");
         } // end else
      } // end else
   } // end else
} // end else
```

İç içe `if` ifadeleri

- Bir çok programcı tarafından aşağıdaki şekilde yazılması tercih edilir

```
1  if (notu >= 90) {
2      puts("A");
3  } // if sonu
4  else if (notu >= 80) {
5      puts("B");
6  } // else if sonu
7  else if (notu >= 70) {
8      puts("C");
9  } // else if sonu
10 else if (notu >= 60) {
11     puts("D");
12 } // else if sonu
13 else {
14     puts("F");
15 } // else sonu
```

Koşullu operatör

- Koşullu operatör(`?:`), `if...else` ifadesi ile yakından ilişkilidir
- Bu operatör C dilinin tek üçlü(ternary) operatörüdür ve üç parametre alır
- Kullanımı `koşul?ifade1:ifade2` şeklindedir
- İfade, koşul doğruysa ifade1'in aksi halde ifade2'nin değerini alır
- `puts((grade >= 60) ? "Geçti" : "Kaldı");`
- a,b ve c değişkenlerinden en büyüğünü x'e atayan koşullu operator kodu nasıl yazılır?

Koşullu operatör

- Koşullu operatör(`?:`), `if...else` ifadesi ile yakından ilişkilidir
- Bu operatör C dilinin tek üçlü(ternary) operatörüdür ve üç parametre alır
- Kullanımı `koşul?ifade1:ifade2` şeklindedir
- İfade, koşul doğruysa ifade1'in aksi halde ifade2'nin değerini alır
- `puts((grade >= 60) ? "Geçti" : "Kaldı");`
- a,b ve c değişkenlerinden en büyüğünü x'e atayan koşullu operator kodu nasıl yazılır?
- `x = a > b ? (a > c ? a : c) : (b > c ? b : c); // ya da`

Koşullu operatör

- Koşullu operatör(`?:`), `if...else` ifadesi ile yakından ilişkilidir
- Bu operatör C dilinin tek üçlü(ternary) operatörüdür ve üç parametre alır
- Kullanımı `koşul?ifade1:ifade2` şeklindedir
- İfade, koşul doğruysa ifade1'in aksi halde ifade2'nin değerini alır
- `puts((grade >= 60) ? "Geçti" : "Kaldı");`
- a,b ve c değişkenlerinden en büyüğünü x'e atayan koşullu operator kodu nasıl yazılır?
- `x = a > b ? (a > c ? a : c) : (b > c ? b : c); // ya da`
- `x = a > b ? a > c ? a : c : b > c ? b : c;`

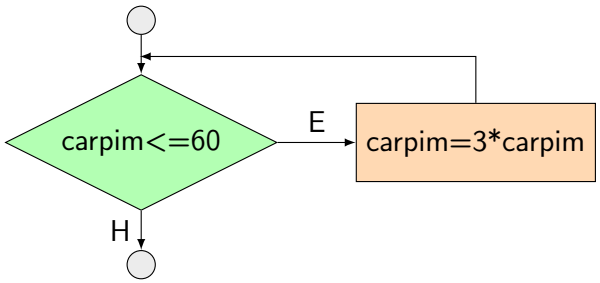
while Döngü Komutu

- Pseudocode

```
Alışveriş listesinde ürün olduğu sürece
        Sıradaki ürünü sepete at ve üzerini çiz
```

- 3'ün 100'den büyük ilk üstü, C kodu

```c
int carpim = 3;
while (carpim <= 100) {
        carpim = carpim * 3;
}
```



Sayaç Kontrollü Döngü

Problem

On öğrencinin bulunduğu bir sınıfta bir sınav yapılmıştır. Notlar(tamsayı ve 0 ile 100 arasında), size verilmiştir. Sınavdan alınan notların ortalamasını bulunuz.

Sayaç Kontrollü Döngü

Pseudocode

1. Toplamı sıfıra eşitle
2. Not sayacını bire eşitle
3. Not sayacı 10'dan küçük veya eşit olduğu sürece
    4. Sıradaki notu al
    5. Notu toplama ekle
    6. Not sayacına bir ekle
7. Ortalama değerini, toplamı ona bölerek hesapla
8. Sınıf ortalamasını yazdır

Sayaç Kontrollü Döngü

```
1  // kod3_2.c
2  // Sayaç kontrollü döngü ile sınıf ortalamasını hesaplayan
   ↪  program
3  #include <stdio.h>
4
5  // main fonksiyonu programın başlangıç noktası
6  int main(void) {
7      // ilk değer atamaları(initialization)
8      int toplam = 0; // not toplamınının ilk değeri sıfır
9      int sayac = 1; // sıradaki not değerinin sayısı/adeti
10
```

Sayaç Kontrollü Döngü

```
11      // İşlem aşaması
12      while (sayac <= 10) { // 10 defa çalış
13          printf("%s", "Notu girin: "); // Girdi için prompt
14          int not_degeri = 0; // not değeri
15          scanf("%d", &not_degeri); // notu kullanıcıdan al
16          toplam = toplam + not_degeri; // notu toplama ekle
17          sayac = sayac + 1; // sayacı artır
18      } // while sonu
19
20      // sonlandırma aşaması
21      int ortalama = toplam / 10; // tamsayı bölmesi
22      printf("Sınıf ortalaması: %d\n", ortalama);
23  } // main fonksiyonu sonu
```

Sayaç Kontrollü Döngü

```
Notu girin: 85
Notu girin: 92
Notu girin: 78
Notu girin: 55
Notu girin: 95
Notu girin: 100
Notu girin: 89
Notu girin: 93
Notu girin: 82
Notu girin: 75
Sınıf ortalaması: 84
```

Gözcü Değerli Döngü

Problem

Program çalıştığında keyfi(sabit olmayan) sayıdaki öğrenci sınavı için sınıf ortalaması hesaplayan programı yazınız.

Gözcü Değerli Döngü

1. Toplamı sıfıra eşitle
2. Sayacı sıfıra eşitle
3. İlk notu al(gözcü olabilir)
4. Eğer kullanıcı gözcü değer girmediyse
    5. Notu toplama ekle
    6. Not sayacına bir ekle
    7. Sıradaki notu al(gözcü olabilir)
8. Sayaç sıfıra eşit değilse
    9. Ortalama değerini, toplamı sayaca bölerek bul
    10. Ortalamayı yazdır
11. Aksi halde
    12. "Hiç not girilmedi" yaz

Gözcü Değerli Döngü

```
1  // kod3_4.c
2  // Gözcü değerli döngü ile sınıf ortalaması hesaplama
3  #include <stdio.h>
4
5  // main fonksiyonu programın başlangıç noktası
6  int main(void) {
7      // ilk değer atamaları(initialization)
8      int toplam = 0; // not toplamınının ilk değeri sıfır
9      int sayac = 0; // sıradaki not değerinin sayısı/adeti
10
11     // İşlem aşaması
12     // İlk notu kullanıcıdan al
13     printf("%s", "Notu girin, sonlandırmak için -1: "); //
       ↪  prompt
14     int not_degeri = 0; // not değeri
15     scanf("%d", &not_degeri); // notu kullanıcıdan al
```

Gözcü Değerli Döngü

```
16
17      // gözcü değer kullanıcıdan okunana kadar döngü çalışsın
18      while (not_degeri != -1) {
19          toplam = toplam + not_degeri;
20          sayac = sayac + 1;
21
22          // Kullanıcıdan sonraki notu al
23          printf("%s", "Notu girin, sonlandırmak için -1: "); //
            ↪  prompt
24          scanf("%d", &not_degeri); // sonraki notu al
25      } // while sonu
26
```

Gözcü Değerli Döngü

```
27      // sonlandırma aşaması
28      // Kullanıcı en az 1 not girdi mi?
29      if (sayac != 0) {
30          // girilen tüm notların ortalamasını hesapla
31          double ortalama = (double) toplam / sayac; // küsuratı
            ↪  sakla
32
33          // Ortalamayı virgülden sonra iki basamak olarak
            ↪  göster
34          printf("Sınıf ortalaması: %.2f\n", ortalama);
35      } // if sonu
36      else {
37          printf("Hiç not girilmedi\n");
38      } // else sonu
39  } // main fonksiyonu sonu
```

Gözcü Değerli Döngü

```
Notu girin, sonlandırmak için -1: 85
Notu girin, sonlandırmak için -1: 92
Notu girin, sonlandırmak için -1: 68
Notu girin, sonlandırmak için -1: 96
Notu girin, sonlandırmak için -1: 89
Notu girin, sonlandırmak için -1: -1
Sınıf ortalaması: 86.00
```

```
Notu girin, sonlandırmak için -1: -1
Hiç not girilmedi
```

Türler Arası Dönüşüm

- 7.2 veya -93.5 gibi değerler küsuratlı(ondalıklı) kısımlar içerir
- Bu değerler `double` türü ile temsil edilebilir
- `toplam / sayac` işleminin sonucu tamsayıdır, çünkü bölme işleminin iki değeri de tamsayıdır
- İfadeyi `(double)toplam / sayac` biçiminde yazmak bölünen toplam değerini `double` türüne dönüştürür
- Bölme işleminde bölünen ya da bölenin herhangi birinin türü `double` ise, işlemin sonucu da `double` olur
- Bu dönüştürme işlemi açık(explicit) dönüşüm ya da casting olarak adlandırılır

Sınav Sonuçlarının Analizi

Bir üniversite, öğrencileri emlak komisyoncuları için eyalet lisans sınavına hazırlayan bir kurs sunmaktadır. Geçen yıl, bu kursu tamamlayan öğrencilerden 10'u lisans sınavına girmiştir. Üniversite, öğrencilerinin sınavda ne kadar başarılı olduğunu doğal olarak bilmek istemektedir. Sonuçları özetleyen bir program yazmanız istenmiştir. Size bu 10 öğrencinin bir listesi verilmiştir. Her ismin yanında, öğrenci sınavı geçtiyse 1, başarısız olduysa 2 yazılacaktır. Programınız sınav sonuçlarını şu şekilde analiz etmelidir:

1. Her sınav sonucunu girin (yani 1 veya 2). Program her sınav sonucu istediğinde "Sonucu girin" mesajını görüntüleyin.
2. Her sınav türünden kaç sonuç olduğunu sayın.
3. Sınavı geçen ve başarısız olan öğrenci sayısını gösteren bir sınav sonucu özeti görüntüleyin.
4. Sınavı sekizden fazla öğrenci geçtiyse, "Öğretmene bonus!" mesajını yazdırın.

Sınav Sonuçlarının Analizi

- Geçenleri sıfıra eşitle
- Kalanları sıfıra eşitle
- Öğrenci sayısını bire eşitle
- Öğrenci sayacı ondan küçük veya eşit olduğu sürece
  - Sıradaki sınav sonucun al
  - Eğer öğrenci geçtiyse
    - Geçenlere bir ekle
  - Aksi halde
    - Kalanlara bir ekle
  - Öğrenci sayacına bir ekle
- Geçen sayısını yazdır
- Kalan sayısını yazdır
- Eğer sekizden fazla öğrenci geçtiyse
  - "Hocaya bonus" yazdır

Sınav Sonuçlarının Analizi

```c
// kod3_6.c
// Sınav sonuçlarının analizi
#include <stdio.h>

// main fonksiyonu programın başlangıç noktası
int main(void) {
    // değişken tanımları ve ilk değer
    ↪ atamaları(initialization)
    int gecen = 0;
    int kalan = 0;
    int ogrenci = 1;

```

Sınav Sonuçlarının Analizi

```
12      // sayaç kontrollü döngü ile 10 defa çalıştır
13      while (ogrenci <= 10) {
14          // Kullanıcı girişi için prompt göster
15          printf("%s", "Sonucu girin (1=geçti,2=kaldı): ");
16          int sonuc = 0; // tek bir sınav sonucu
17          scanf("%d", &sonuc);
18
19          if (sonuc == 1) {
20              gecen = gecen +1;
21          } // if sonu
22          else { // aksi halde kalanı artır
23              kalan = kalan + 1;
24          } // else sonu
25
26          ogrenci = ogrenci + 1; // öğrenci sayısını artır
27      } // while sonu
28
```

Sınav Sonuçlarının Analizi

```
29      // sonlandırma aşaması; geçen ve kalan sayılarını göster
30      printf("Geçen %d\n", gecen);
31      printf("Kalan %d\n", kalan);
32
33      // sekizden fazla geçen varsa, "Hocaya bonus!"
34      if (gecen > 8) {
35          puts("Hocaya bonus!");
36      } // if sonu
37  } // main fonksiyonu sonu
```

Sınav Sonuçlarının Analizi

```
Sonucu girin (1=geçti,2=kaldı): 1
Sonucu girin (1=geçti,2=kaldı): 2
Sonucu girin (1=geçti,2=kaldı): 1
Sonucu girin (1=geçti,2=kaldı): 1
Sonucu girin (1=geçti,2=kaldı): 2
Sonucu girin (1=geçti,2=kaldı): 2
Sonucu girin (1=geçti,2=kaldı): 2
Sonucu girin (1=geçti,2=kaldı): 1
Sonucu girin (1=geçti,2=kaldı): 1
Sonucu girin (1=geçti,2=kaldı): 1
Geçen 6
Kalan 4
```

Sınav Sonuçlarının Analizi

```
Sonucu girin (1=geçti,2=kaldı): 1
Sonucu girin (1=geçti,2=kaldı): 2
Sonucu girin (1=geçti,2=kaldı): 1
Sonucu girin (1=geçti,2=kaldı): 1
Sonucu girin (1=geçti,2=kaldı): 1
Sonucu girin (1=geçti,2=kaldı): 1
Sonucu girin (1=geçti,2=kaldı): 1
Sonucu girin (1=geçti,2=kaldı): 1
Sonucu girin (1=geçti,2=kaldı): 1
Sonucu girin (1=geçti,2=kaldı): 1
Geçen 9
Kalan 1
Hocaya bonus!
```

Atama Operatörleri

```
int c = 3, d = 5, e = 4, f = 6, g = 12;
```

Atama operatörü	Örnek ifade	Açıklama	Atama
+=	c += 7	c = c + 7	c'ye 10
-=	d -= 4	d = d - 4	d'ye 1
*=	e *= 5	e = e * 5	e'ye 20
/=	f /= 3	f = f / 3	f'ye 2
%=	g %= 9	c = c % 9	g'ye 3

Artırma ve Azaltma Operatörleri

Operatör	Örnek ifade	Açıklama
`++`	`++a`	Bir arıtr ve sonra yeni değeri ifade olarak kullan
`++`	`a++`	Mevcut değeri ifade olarak kullan, sonra bir artır
`--`	`--a`	Bir azalt ve sonra yeni değeri ifade olarak kullan
`--`	`a--`	Mevcut değeri ifade olarak kullan, sonra bir azalt

Önceden ve Sonradan Artırma

```c
// kod3_7.c
// Önde artırmak ve sonda artırmak
#include <stdio.h>

// main fonksiyonu programın başlangıç noktası
int main(void) {
    // sonradan artırımı gerçekleştir
    int c = 5; // c'ye 5 ata
    printf("%d\n", c); // 5 yazar
    printf("%d\n", c++); // 5 yazıp artırır
    printf("%d\n\n", c); // 6 yazar

    // önceden artırımı gerçekleştir
    c = 5; // c'ye 5 ata
    printf("%d\n", c); // 5 yazar
    printf("%d\n", ++c); // artırıp 6 yazar
    printf("%d\n\n", c); // 6 yazar
} // main fonksiyonu sonu
```

Önceden ve Sonradan Artırma



İşlem Önceliği

Operatörler	Gruplama	Tür
++ (son ek) -- (son ek)	sağdan sola	son ek
+ - (tip) ++ (ön ek) -- (ön ek)	sağdan sola	tekli (unary)
* / %	soldan sağa	çarpımsal
+-	soldan sağa	toplamsal
< <= > >=	soldan sağa	ilişkisel
== !=	soldan sağa	eşitlik
?:	sağdan sola	koşul
= += -= *= /= %=	sağdan sola	atama